Semi-Automatic Object-Oriented Software Design Using Metaheuristic Algorithms

Zeynab Javidi
Software Engineering Lab
Department of Computer Engineering and IT
Shiraz University of Technology
Shiraz, Iran
Z.Javidi@sutech.ac.ir

Reza Akbari
Software Engineering Lab
Department of Computer Engineering and IT
Shiraz University of Technology
Shiraz, Iran
akbari@sutech.ac.ir

Omid Bushehrian
Department of Computer Engineering and IT
Shiraz University of Technology
Shiraz, Iran
bushehrian@sutech.ac.ir

***Abstract*—The quality of software design always has a significant impact on the extendibility and maintainability of the final product. Automatic techniques may help designers to achieve better design. There are several ways for software design automation. Generally Search-based methods such as GA, ant colony, and ICA are used for problems with large search space in which finding the optimal solution is hard. In this paper a hybrid algorithm called ICA-TS (Imperialist Competitive Algorithm-Tabu Search) is presented to generate class diagram of the under design system automatically. The method has three phases: First, formal concept analysis (FCA) for preprocessing phase of the method is used as a mean to generate initial solution. Next a hybrid of ICA and TS is used to update solutions. The relationships between classes are determined in third phase. Three standard case studies are used for performance evaluation and the results are compared with results of genetic and simple ICA. The results show that the presented method has competitive results and it can generate more efficient class diagram in terms of cohesion, coupling and complexity of system.**

Keywords-Software Design; Search-Based Software Engineering; Meta-Heuristic Algorithm; Formal Concept Analysis.

I. INTRODUCTION

Software design is one of the main steps in software development process. In this step the system is modeled before coding and its internal structure and behavior are specified precisely. There are several problems in software design where each of them has its own activities. Assigning methods and attributes to the classes in an object oriented system which is known as Class Responsibility Assignment (CRA) is an important and complex task [1]. IT aims to find the best assignment of responsibilities of the classes according to some design metrics such as coupling and cohesion [2]. Automated CRA is known as a complex and difficult task, and it is belong to the class of NP-Hard problems [3], [4] because there are so many solutions that could be found for it. Therefore using an automated method that uses meta-heuristics can decrease dependence on human judgment and decision making as well as the required time for designing the target software [1]. In general, this type of algorithms has the ability to help the designers to produce faster and more efficient solutions.

CRA Problem generally can be divided into two major phases: assigning responsibilities to classes and setting relationships between these classes [1]. Recently some researchers [3], [5], [6], [7] have proposed several meta-heuristic algorithm like genetic algorithm (GA), Particle Swarm Optimization (PSO) and Ant Colony Optimization (ACO) and the model based on clustering method for solving CRA problem [1]. These studies showed that meta-heuristic method have good performance in solving CRA problem. The capability of these methods in solving combinatorial optimization problem encouraged us to use them for assigning responsibilities to the classes in the design process of an object oriented system.

In this paper a three phase method based on search-based algorithms is proposed for solving CRA problem. At the first phase, formal concept analysis is used to extract the initial solution from the analysis documents and models. Next, at the second phase, ICA is combined with TS to improve the initial solutions and obtain the best possible solution through iterations. Finally, at the third phase, relationships between elements of the design model are determined. The proposed method is compared with the methods. The experimental results show that the presented method is efficient and has the ability to produce competitive results in comparison with the other methods.

The remaining of the paper is organized as follows: some basic concepts and a review of the related works are presented in Section II. The details of the proposed method are given in Section III. The performance analysis is given in Section IV. Finally Section V concludes this work.

978-1-5090-4330-9/17/$31.00 ©2017 IEEE

II. RELATED WORKS

A. Search based software engineering

Search-Based software engineering has become one of the most interesting fields in recent years [20]. Search-Based techniques play an important role to solve a wide range of problems especially software engineering problems. The first step in solving problem with search-based techniques is formulating the problem at hand as a search problem. Generally, evolutionary and meta-heuristic algorithms work as follows: first an initial random solution is generated, and then the method tries to improve these initial solutions with its own techniques in a repeated loop. The terminate condition for algorithm is depends on kind of problem.

Using search methods for solving design problems is known as search based software design. As an example of using search methods for object-oriented software design automation, Amoui et al. [8] have used genetic algorithms. In their work, reusability of software system with applying design pattern was considered.

In the works presented by Simones and Parmee in [9] [10] [11], the use case of system was used as the initial point. Afterwards, data items are assigned to features and operation are assigned to methods. Then a set of uses has been defined between two sets. The class concept was used to group methods and features. A design solution is a chromosome and mutation on a chromosome is performed with moving methods and features from one class to another. This model gets requirements of system as the input and then generates a complete architecture design that it can be harder than when a prepared system exist.

B. Search based responsibility assignment

The CRA problem as a part of software design can be formulated as a search problem. The CRA is an early stage activity of software design and has positive effect on the quality of design [4]. Successful responsibility assignment results an efficient and high quality design of the software. Bowman et al. [3] has defined the CRA problem as the activity of assigning operations and attributes they manipulate to the right classes.

In recent years, different types of search methods such as multi-objective genetic algorithm (MOGA) [12], [13], [18], Strength Pareto Approach (SPEA2) [14], GA [13], [15], [16], [17], [19], Hill Climbing (HC), Simulated Annealing (SA) [13], and Particle Swarm Optimization (PSO) [13], Ant Colony Optimization (ACO) [7] have been presented by authors to cope with CRA problem. Some of these methods such as MOGA and SPEA2 considered multi-objective CRA while the other ones were single objective. In these works, CRA problem has been modeled as an optimization problem and appropriate cost function based on design metrics has been defined. After that, the aforementioned search methods haven applied to obtain the best solution for the problem.

In CRA problem, the responsibilities and different types of dependencies should be considered for assignment process. In object-oriented design, methods and attributes are known as responsibilities and dependencies between them can be modeled as data, functional, and semantic [1], [3], [5]. In data and functional dependencies which are known as structural dependencies, an attribute depends to a method and one method depends to another method respectively. The semantic dependency is known as a dependency where methods and attributes depend to each other based on semantic concepts.

For solving CRA problem we have to create an initial class diagram that it is an assignment of responsibility to classes. Suppose n is the number of responsibilities of the system so any possible assignment of n responsibilities to classes is a sample class diagram or a candidate solution for problem. As mentioned before, although there is a huge search space for problem not all of the assignments are valid. Therefore with applying search-based algorithms the search space can be limited and by using suitable design metrics search can be lead to an optimal solution. The number of class for assignments is unknown from beginning so in the proposed method we have used formal concept analysis as the first phase of creating several initial assignments with a specific number of classes.

III. PROPOSED METHOD

In this paper, a new method based on meta-heuristic algorithms is presented for semi-automatic design of object oriented software. The proposed method takes documentations of the analysis phase consist of use cases, attributes and methods as input and produces a class diagram of system as the output. The model has three main phases:

1) Performing an initial raw assignments using formal concept analysis,

2) Optimizing this initial assignment using a hybrid search-based algorithm called ICA-TS,

3) Setting relationships between classes.

The first and third phases are done semi-automatically and the second phase is done automatically. A schematic view of the proposed method is shown in Fig. 1. The details of these phases are given below. It should be noted that representation of the solutions and the cost function are important in designing a search-based method. Hence, it is necessary to describe them precisely.

A. Solution Representation

One of the first steps in designing search based method is selecting the best way to represent the solution. Presenting appropriate representation has positive impact on the performance of the search method. For CRA problem, different ways have been proposed for presenting solutions in literature. For example, two solution representation methods have been presented by Simons and Smith [21]. In the first method, which is called Naïve Grouping (NG), a solution is encoded as a vector of d responsibilities. The value of each responsibility is selected from set $\{1, 2\dots C\}$, where C is the maximum number of classes in the design model. Each integer in the solution vector shows an assignment $a= j$, interpreted as assignment of element $i \in \{1, 2\dots d\}$ to the class $j \in \{1, 2\dots C\}$.

The second representation which is called Extended Permutation (XP) is inspired by TSP and Vehicle Routing Problem (VRP). In XP representation, candidate solutions are

represented as permutations of a set of $(d+C-1)$ element [21], where d is number of responsibilities and C is number of classes. Each integer number in permutation with regards to first number between d and C shows that responsibility belong to that class. In this paper XP representation is used.

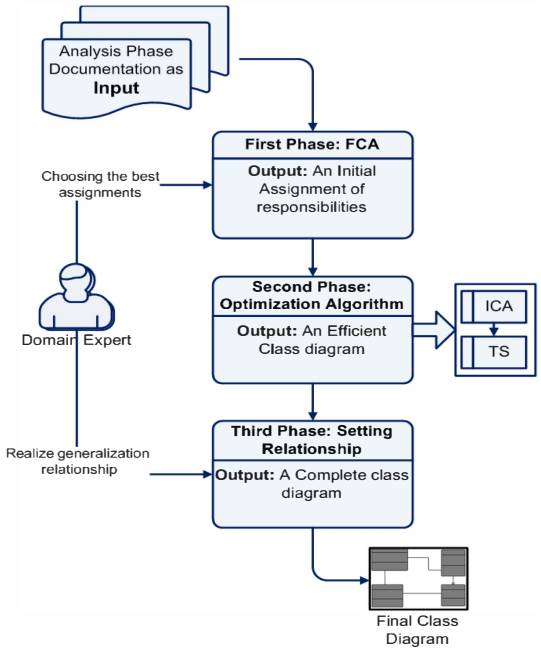

Figure 1. Schematic view of the proposed method

B. *Optimization Metric*

Using object-oriented metrics is an efficient way to evaluate the quality of an architectural design method. Most of these metrics have been introduced by Lanza et al. [22] and some of them also have been introduced by chidamber and kemerer [23]. Generally there are three categories of object-oriented metrics: coupling between classes, inner cohesion of the class and complexity of design [20]. In this paper we have used these three metrics and we follow the approaches introduced by vali tawosi et al. [7] to calculate them.

To calculate the cost function, weighted combination of four criteria is computed. The cohesion metric has to be maximized and the other three should be minimized. Cost function should be minimized. Hence relation composed of combination of minimization metrics and reverse of maximization metric.

$$CostFunction(D) = w_1 Cop(D) + w_2 Complexity(D) + w_3 ClassSizeStandardDeviation(D) + \frac{w_4}{Coh(D)} \quad (1)$$

Where:

$$\omega 1 + \omega 2 + \omega 3 + \omega 4 = 1 \quad (2)$$

In equation (1), Cop (D), Complexity (D), Coh (D) and ClassSizeStandardDeviation (D) show coupling, complexity of design, cohesion, and standard deviation of the class size. The right choice of weights depends on design priorities. Also they can be chosen empirically.

C. *First phase: Formal Concept Analysis*

As shown in Fig. 1, by receiving the documentation of the analysis phase as the input, the Formal Concept Analysis (FCA) is applied on them. In this paper the purpose of using formal concept analysis is extracting the initial candidate class using review concept lattice. This is done semi-automatically by an expert opinion.

FCA provides the ability to model the conceptual relationships using lattice diagrams [24]-[26]. In FCA, formal things and formal features are used and the relations between them are created. Each relation shows that each formal thing has a formal feature. The details of FCA have been given in [25] and [26]. For responsibility assignment, use cases are considered as formal things and Responsibilities as formal features [26]. Using Formal Concept Analysis we have a context that consists of some concepts. These concepts can be candidate classes. The context for CBS case study depicted in Fig. 2. Each row is a use case and each column is a responsibility in the system and each mark shows the relation between them. Lattice diagram for this context is shown in Fig. 3. Each node in this lattice is considered as the concepts or the candidate classes.

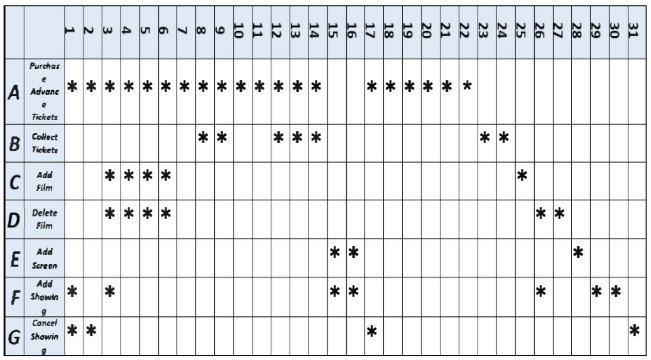

Figure 2. Context for CBS Case Study

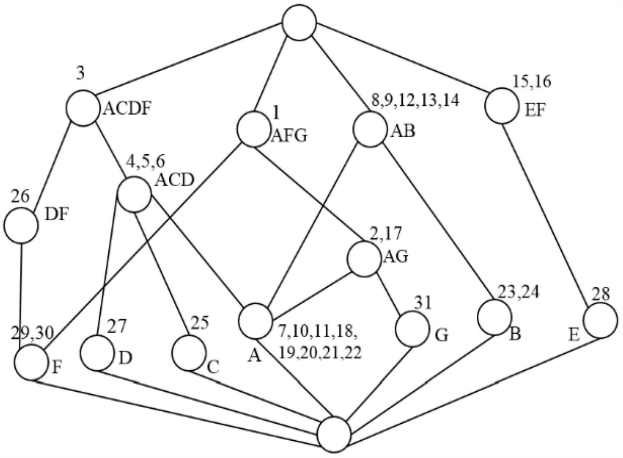

Figure 3. Concept Lattice Related to CBS Context

Then an initial assignment is created with help of a domain expert. This initial assignment is used as the input of the next phase. At the next phase, the proposed method tries to optimize the initial solutions using a hybrid method based on Imperialist Competitive Algorithm and Tabu Search.

D. Second phase: Optimization Algorithm

The algorithm used for optimization combines two algorithms Imperialist Competitive Algorithm and Tabu Search and we called it ICA-TS. First, the assignment generated by previous step is given to ICA as initial solution (Input), and then final solution generate by ICA give to TS as input and this algorithm can explore the search space more. As mentioned earlier, several initial solutions are generated from previous phase by an expert opinion and each of them is considered as an input to this phase separately. Finally the output of this phase is compared and the best answer in terms of mentioned metrics is selected. The pseudo-code of the presented ICA-TS is given in Fig. 4.

ICA Phase

1. Initialization of the algorithm. Create initial empires (based on FCA output).
2. Assimilation: Colonies move towards their imperialist in different directions.
3. Revolution: Random changes occur in some countries. Swap, reversion or insertion is applied to candidate solution.
4. Position exchange between a colony and Imperialist. A colony with a better position than the imperialist has the chance to take the control of empire by replacing the existing imperialist.
5. Imperialistic competition: All imperialists compete to take possession of colonies of each other.
6. Eliminate the powerless empires. Weak empires lose their power gradually and they will finally be eliminated.
7. If the stop condition is satisfied, stop, if not go to step 2.

TS Phase

8. Current solution = best solution of ICA
9. Best solution = current solution
10. Accept transaction = 0
11. While (accept transaction == 0)
    12. Generate transaction
    13. If move ∉ Tabu list
        14. Accept_ transaction = 1
        15. Update Tabu list
        16. Update current solution
    17. Else
        18. Accept_ transaction = 0
19. End while
20. If current solution is better than best solution
21. Best solution =current solution
22. If the stop condition is satisfied, stop, if not go to step 8.
23. Report best solution

Figure 4. Pseudo-code for ICA-TS algorithm

In step 1 each Initial solution generated in FCA phase is permutated for generate an empire. For assigning colonies to imperialist roulette wheel selection is used. In this step a parameter which called alpha is used that it is selection pressure. If alpha=0 then each imperialist has same chance for possession of colonies, and if alpha=1 then best imperialist has more chance. We set alpha=1. In step 2 each colonies move toward its imperialist with relation (3) in which Parameter Beta determine amount of movement of colonies toward imperialist which is between 0 and 2. Parameter n is length of solution. For Termination condition we use constant iteration in which the algorithm has no improvement any more.

$$col(i).position = col(i).position + beta * random(n) * (Imp.position - col(i).position) \quad (3)$$

Initial solution in TS Phase is the best solution generated in ICA phase. This phase can explore search space more than ICA and can converge to best solution faster because it start from better position in search space. TS is based on create diversity in search process using Tabu list. This list contain several recently used action and algorithm can't use this action unless it generate the best solution ever found.

E. Third phase: Setting Relationship

A class diagram consists of some classes and their relationship. All types of relationship between classes are: association, generalization, dependency and abstract/realization [27]. Here, we just consider association, generalization and dependency relationships.

A dependency is a type of relationship between two elements of a design model where an element depends on the other one. Two elements are dependent if a change in one element affects the other element [1]. The association relationship is a connection between classes. To set association relationships between elements of the design model, the method proposed by Bowman et al. [3] is used here.

For generalization which is implemented by inheritance in programming languages, the method presented by Masoud et al. [1] is used here. This method which is based on some heuristics given in [28] identifies the generalizations in three steps.

IV. Experimental Study

In this section, the performances of the proposed methods are analyzed. First case studies are described and the settings of the algorithms are presented. Next, experimental results are given in details.

A. Case studies and parameters' settings

To evaluate the proposed method, three case studies [21] are used. TABLE I shows brief description of problems used here. Cinema Booking System (CBS) and Graduate Development Program (GDP) are small-scaled and Select Cruises (SC) is a middle-scaled problem.

The performances of two versions of the proposed method (i.e. ICA-TS and ICA-TS with FCA) are compared with the genetic algorithm (GA) and ICA. For Implementing ICA and GA we used 350 iterations for CBS and GDP and 500 iterations for SC. For TS step we used 30 iteration for CBS and GDP and 40 for SC.

Revolution operation in ICA is implemented like mutation operation in GA with three operation swap, reversion and insertion. Generation number for GA and country number for ICA is intended 100. Size of Tabu list in TS is set 0.4 of all possible action. Weights for metric value given in relation 2 are set as follows:

$$w_1=w_2=w_4=0.2 \text{ and } w_3=0.4$$

The values for these weights are obtained experimentally.

TABLE I. CASE STUDIES

Title	Number of				
	Attributes	Methods	Responsibilities	Classes	Usage
Cinema Booking System (CBS)	16	15	31	5	39
Graduate Development Program (GDP)	43	12	55	5	107
Select Cruises (SC)	62	30	108	16	141

B. *Experimental results*

The results of applying GA, ICA, ICA-TS, and ICA-TS with FCA are presented in TABLES II, III, and IV. Each algorithm is applied for 10 times and the average results are reported. Also, the results are compared with a design proposed by an expert designer. The Cohesion metric must be maximized and the three other metrics and cost function must be minimized.

It can be seen from the tables that the results for automatic methods are better than manual design in all cases. Also results are better for ICA-TS in most cases. In small case studies, these algorithms have competitive results. For systems with larger size (i.e. Select Cruises case study) performance of the ICA-TS is better than the others. However, ICA-TS is not the best in terms of runtime.

TABLE II. METRIC VALUE FOR CBS CASE STUDY FOR 3 ALGORITHM

Method		Criteria				
		Cost Function	Cohesion	Coupling	Complexity	CSSD
GA	Best	0.497	0.55045	0.31579	0.29446	0.03148
	Avg.	0.526	0.53576	0.35120	0.30468	0.05453
	St.D.	0.030	0.01618	0.03935	0.01602	0.04722
ICA	Best	0.497	0.55045	0.28230	0.26221	0.03148
	Avg.	0.512	0.53780	0.33481	0.29664	0.03422
	St.D.	0.015	0.01291	0.03916	0.01587	0.00822
ICA-TS	Best	0.497	0.55045	0.31579	0.29446	0.03148
	Avg.	0.502	0.54827	0.32632	0.29774	0.03148
	St.D.	0.012	0.00572	0.03158	0.00983	0.00000
With FCA	Best	0.469	0.54072	0.25000	0.15292	0.03099
	Avg.	0.473	0.53570	0.25849	0.17834	0.03099
	St.D.	0.007	0.01288	0.00594	0.00934	0.00000
Expert		0.886	0.27870	0.33851	0.25631	0.12592

TABLE III. METRIC VALUE FOR GDP CASE STUDY FOR 3 ALGORITHM

Method		Criteria				
		Cost Function	Cohesion	Coupling	Complexity	CSSD
GA	Best	0.505	0.503	0.360	0.143	0.000
	Avg.	0.530	0.497	0.405	0.191	0.000
	St.D.	0.025	0.014	0.048	0.026	0.027
ICA	Best	0.505	0.503	0.360	0.143	0.000
	Avg.	0.519	0.493	0.375	0.176	0.005
	St.D.	0.019	0.019	0.031	0.020	0.015
ICA-TS	Best	0.505	0.503	0.360	0.143	0
	Avg.	0.510	0.498	0.364	0.174	0
	St.D.	0.011	0.014	0.006	0.010	0
With FCA	Best	0.482	0.504	0.271	0.157	0.017
	Avg.	0.486	0.504	0.271	0.157	0.017
	St.D.	0.003	0.000	0.000	0.000	0.000
Expert		1.1304	0.215	0.458	0.164	0.198

TABLE IV. METRIC VALUE FOR SC CASE STUDY FOR 3 ALGORITHM

Method		Criteria				
		Cost Function	Cohesion	Coupling	Complexity	CSSD
GA	Best	0.559	0.526	0.294	0.350	0.078
	Avg.	0.593	0.518	0.347	0.387	0.149
	St.D.	0.019	0.010	0.030	0.026	0.047
ICA	Best	0.558	0.518	0.365	0.332	0.037
	Avg.	0.587	0.495	0.407	0.344	0.080
	St.D.	0.018	0.017	0.022	0.008	0.023
ICA-TS	Best	0.519	0.532	0.252	0.337	0.034
	Avg.	0.536	0.523	0.325	0.349	0.064
	St.D.	0.017	0.014	0.042	0.014	0.041
With FCA	Best	0.5099	0.5305	0.2730	0.3001	0.0269
	Avg.	0.5112	0.5275	0.2917	0.3034	0.0325
	St.D.	0.0012	0.0016	0.0101	0.0017	0.0033
Expert		1.2686	0.333	0.542	0.370	0.141

The experimental results for SC case study are presented in Fig. 5. Notice that only cohesion metric must be higher. Diagram shows that the ICA-TS with FCA provide better performance in comparison with the other methods.

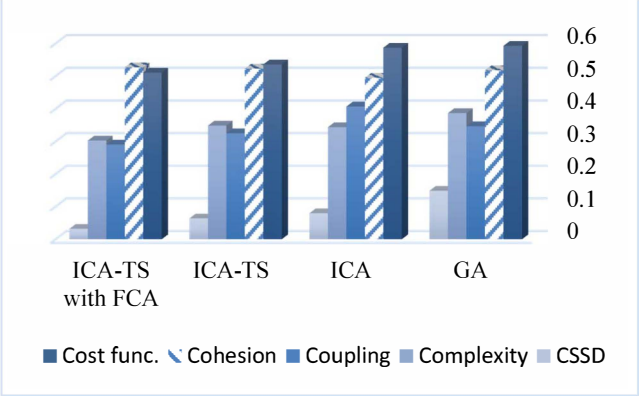

Figure 5. Camparison between 4 Algorithms for SC case study for 10 independent run

Fig. 6 shows the manual design presented by an expert and Fig. 7 shows the best automated design for CBS case study obtained by ICA-TS with FCA. The number of suggested classes by the presented method is 4 while the manual design results a class diagram with 5 classes. Also, the assigned responsibilities to the classes are different in these two models.

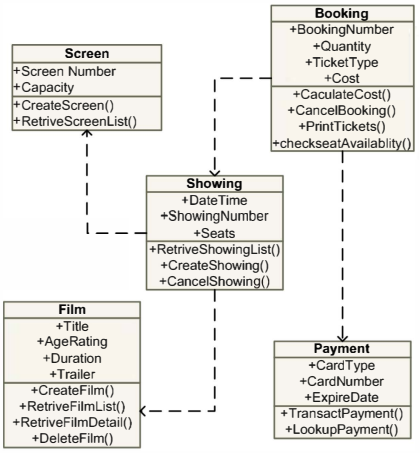

Figure 6. Manual Design for CBS case study

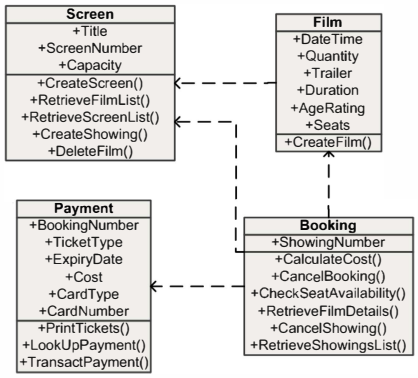

Figure 7. Automatic design obtained by the proposed method for CBS case study

V. CONCLUSION AND FUTURE WORKS

In this paper the automated software design problem was considered and three meta-heuristic methods were used to solve the problem. The proposed method has three main phases including FCA, Optimizing algorithm and setting relationships.

Analysis documentation was used as the input of our method and finally it generated an optimal class diagram in terms of object-oriented metrics. Three different meta-heuristic algorithms i.e. GA, ICA and ICA-TS for optimization phase were used. ICA-TS is a hybrid algorithm presented here by combining two algorithm Imperialist Competitive Algorithm (ICA) and Tabu Search (TS). Also, Formal Concept Analysis (FCA) was used to extract initial assignment of responsibilities to classes. Simulation results showed that the hybrid algorithm had better results and generated more efficient class diagram. Moreover its performance was better for larger systems. Using FCA also affected the performance of final class diagram and it helped algorithms to converge faster.

As the future work, we aim to apply some GOF design patterns [15] as standard design solutions to improve the automated design quality.

REFERENCES

[1] Masoud hamid, Jalili Saeed, "A clustring based model for class responsibility assignment problem in object-oriented analysis," The journal of systems and software, vol. 93, pp. 110-131, 2014.

[2] Briand L.C., Daly J.W. Wust, J., "A unified framework for cohesion measurement in object-oriented systems," Emprical Software Engineering 3 (1), pp. 57-117, 1997.

[3] Michael Bowman, Yvan Labiche, "Solving the Class Responsibility Assignment Problem in Object-Oriented Analysis with Multi-Objective Genetic Algorithms," IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, vol. 36, no. 6, pp. 817-837, 2010.

[4] Larman, C., Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, Prentice Hall, 2004.

[5] Simons, C.L., Parmee, I.C., Gwynlly,R, "Interactive Evolutionary Search in Upstream Object-Oriented class design," IEEE Transaction on Software Engineering , vol. 36, no. 6, pp. 798-816, 2010.

[6] Goran Glavas, Kresimir Fertalj, "Metaheuristic Approach to Class Responsibility Assignment Problem," in Information Technology Interfaces (ITI), 2011.

[7] Vali Tawosi , Saeed Jalili , S.M. Hossein Hasheminejad, "Automated Software Design using Ant Colony Optimization with Semantic Network Support," The Journal of Systems & Software, vol. 109, pp. 1-17, 2015.

[8] M. Amoui, S.Mirarab, S. Ansari and C. Lucas, "A genetic algorithm approach to design evolution using design pattern transformation," International Journal of Information Technology and Intelligent Computing, vol. 1, 2006.

[9] C.L. Simons, I.C. Parmee, "(a) Single and multi-objective genetic operators in object-oriented conceptual software design," in Proceedings of the Genetic and Evolutionary Computation Conference, GECCO'07 pp. 1957–1958, 2007.

[10] C.L. Simons, I.C. Parmee, "(b) A cross-disciplinary technology transfer for search-based evolutionary computing: from engineering design to software engineering design," Engineering Optimization, vol. 39, no. 5, p. 631–648, 2007.

[11] C.L. Simons, I.C. Parmee, "User-centered, evolutionary search in conceptual software design," in Proceedings of the IEEE Congress on Evolutionary Computation, CEC'08, pp. 869–876, 2008.

[12] M. Bowman, L.C. Briand, Y. Labiche, "Solving the class responsibility assignment problem in object-oriented analysis with multi-objective genetic algorithms," in Technical report SCE-07-02, Carleton University, 2008.

[13] A. Engelbrecht, Computational Intelligence: An Introduction 2nd ed, John Willey & Sons, 2007.

[14] Zitzler, E., Laummans, M. Thiele, L., "SPEA2: Improving the Stregth Pareto," Evolutionary Algorithm, no. 103, pp. 95-100, 2001.

[15] O. Raiha, K. Koskimies, E. Makinen, "(a) Genetic synthesis of software architecture," in Proceedings of the 7th International Conference on Simulated Evolution and Learning, SEAL'08, Australia, in: LNC, 2008.

[16] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides and Grady Booch, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 2003.

[17] O. Raiha, K. Koskimies, E. Makinen, T., "(b) Pattern based genetic model refinements in MDA Systa," Nordic Journal of Computing , vol. 14 , no. 4, p. 338–355, 2008.

[18] O. Raiha, K. Koskimies, E. Makinen, "Scenario-based genetic synthesis of software architecture," in Proceedings of the 4th International Conference on Software Engineering Advances, ICSEA'09, Portugal, IEEE Computer Society Press, 2009.

[19] Outi Raiha, Erkki Makinen, "Generating Software Architecture Spectrum with Multi-Objective Genetic Algorithms," in Third World Congress on Nature and Biologically Inspired Computing, 2011.

[20] O. Raiha, "A Survey on Search Based Software Design," journal of Computer Science Review ,Elsevier Science Publishers, 2010.

[21] C. L. Simons, J. E. Smith, "A comparison of meta-heuristic search for interactive software design," Soft Computing, vol. 17, no. 11, p. 2147–2162, 2013.

[22] Lanza,M.,Marinescu,R.,Ducasse,S., Object-Oriented Metrics inPractice: Using Software Metrics to Characterize, Evaluate, and Improve the Design of ObjectOriented Systems, Springer-VerlagBerlinHeidelberg, 2006.

[23] Shyme R. Chidamber, Chris F. Kemerer, "A Metrics Suite for object Oriented Design," IEEE Transaction on Software Engineering , vol. 20, no. 6, pp. 476-493, 1994.

[24] Stephan Düwel, Wolfgang Hesse, "Bridging the gap between Use Case Analysis and Class Structure Design by Formal Concept Analysis," Proceedings of Modellierung, pp. 27-40, 2000.

[25] Ganter, Bernhard, Wille, Rudolf, Formal Concept Analysis Mathematical Foundations, Springer -Verlag Berlin Heidelberg, 1999.

[26] Stephan Düwel, Prof. Dr. Wolfgang Hesse, Identifying Candidate Objects during System Analysis.

[27] Hans-Erik Eriksson, Magnus Penker, Brian Lyons, David Fado, UML 2 Toolkit, Wiley Publishing, 2003.

[28] A. Riel, Object-Oriented Design Heuristics, Boston, MA, USA: Addison-Wesley Longman Publishing Co., 1996.